

C# Enumerations

C# Enumerations

- An enumeration is a special kind of value type limited to a restricted and unchangeable set of numerical values.
- By default, these numerical values are integers, but they can also be longs, bytes, etc. (any numerical value except char) .
- If you are creating an employee payroll system, you may wish to use the constants VP, Managers, Officer, Peon rather than raw numerical values such as {0,1,2,3}.
- C# supports the notion of custom enumerations for this vary reason.

```
enum EmpType
```

```
{
```

```
    VP, // =0
```

```
    Managers, // =1
```

```
    Officer, // =2
```

```
    Peon // =3
```

```
}
```

C# Enumerations

```
public static void AskForBouns(EmpType e)
{
    switch (e)
    {
        case EmpType.VP
            Console.WriteLine("You already get enough cash!");
            break;
        case EmpType.Manager
            Console.WriteLine("Next Year!");
            break;
        case EmpType.Officer
            Console.WriteLine("You get 20% cash extra!");
            break;
        case EmpType.Peon
            Console.WriteLine("You get 100% bonus cash!");
            break;
    }
}
```

C# Enumerations

```
public static int Main (string[] args)
{
    // Make a Poen type
    EmpType etype;
    etype = EmpType.Poen
    AskForBouns(etype);
    return 0;
}
```

C# Enumeration Example

```
public enum DAYS
{
    Monday, Tuesday, Wednesday, Thursday, Friday,
    Saturday, Sunday
}
```

The numerical values are set up according to the following two rules:

1. For the first literal: if it is unassigned, set its value to 0.
2. For any other literal: if it is unassigned, then set its value to one greater than the value of the preceding literal.

C# Enumeration Example

```
public enum DAYS
{
    Monday, Tuesday, Wednesday, Thursday, Friday,
    Saturday, Sunday
}
```

- **DAYS.Monday will be set to 0**
- **The values increased until DAYS.Sunday is set to 6.**
- **The values specified in an enumeration are static, so we have to refer to them in code using the name of the enumeration: "DAYS.Monday" rather than just "Monday".**
- **These values are final - you can't change their runtime value.**

C# Enumeration Example

```
public enum DAYS
{
    Monday=1, Tuesday, Wednesday, Thursday, Friday,
    Saturday, Sunday
}
```

C# Enumeration Example

```
public enum DAYS  
{  
    Monday=1, Tuesday, Wednesday,  
    Thursday=1, Friday, Saturday, Sunday  
}
```


C# Enumeration Advantage

- In C# enumerations are type-safe.
- For instance, the following code should not compile:
`int i = DAYS.Monday;`
- In order to get this code to compile, you would have to make explicit casts both ways (even converting from DAYS to int)
`int i = (int) DAYS.Monday;`

C# Enumeration Changing Underlying Type

- In C# enumerations, the storage type used for each item in an enumerations automatically maps to an integer by default.
- However you are free to change this to your liking.
- For example

```
enum EmpType : byte
{
    VP, // =0
    Managers, // =1
    Officer, // =2
    Peon // =3
}
```

System.Enum Base Class

- C# enumerations, they implicitly derive from System.Enum base class.
- System.Enum contains various important methods

GetUnderlyingType()

Enum.GetUnderlyingType(typeof(EmpType))

GetValues()

Array obj = Enum.GetValues(typeof(EmpType));